

Part 2: Written Explanation

StockHive — Susesesh SJ — Engineer (Platform) Take-Home Assignment

StockHive is structured around a clear separation between core infrastructure and feature modules. The core folder holds services, models, and guards that any part of the app can depend on, while each screen — login, dashboard, product form — lives in its own feature folder as a standalone component. This keeps the codebase easy to navigate for a small, focused app, and avoids the overhead of a shared NgModule that this scope doesn't need. The one place I was deliberate about structure was the stock adjustment logic: rather than writing two separate Firestore calls (update the product, then log the movement), I wrapped both in a single Firestore transaction. This guarantees the quantity and the audit trail never drift apart, even if a write partially fails.

If this were going into production for 10,000 users a day, several things would need to change. The client-side search and filtering would need to move to proper Firestore queries with composite indexes, since pulling the full product list into the browser stops scaling past a few thousand documents. I'd also introduce pagination on the dashboard, tighten the Firestore security rules to scope reads/writes to a user's own organization rather than any authenticated user, and move stock-adjustment logic into a Cloud Function so business rules aren't only enforced client-side. Monitoring, structured logging, and automated tests would also need to be added before this could be trusted at that scale.

Given the 48-hour window, the thing I left out is automated testing. Right now correctness is verified by manually walking through each feature on the deployed app, which is fine for a scoped demo but wouldn't hold up over time. With more time, I'd start with unit tests around the stock adjustment transaction, since that's the piece with real business logic and the highest cost if it silently breaks.

Beyond the core requirements, I used Angular Signals for local UI state (search term, filter toggle) instead of plain component properties, since they make change detection more predictable and read cleanly. I also added light CSS transitions on the login and dashboard screens purely for polish — these don't affect functionality but make the app feel more finished for a review context.